

UMBC Interactive Accessibility Map

Software Requirements Specification (SRS)

Version 1.0

9/16/2025

Alex Gallagher, Jack Xu, Ben Currie



UMBC

Table of Contents

Software Requirements Specification (SRS).....	1
Table of Contents.....	2
1. Introduction.....	4
1.1. Purpose.....	4
1.2. Scope.....	4
1.3. Intended Audience.....	4
1.4. Definitions.....	5
1.5. References.....	5
2. Overall Description.....	6
2.1. Description.....	6
2.2. User Classes.....	6
2.3. Operating Environment.....	7
2.4. Design and Implementation Constraints.....	7
2.5. Assumptions.....	7
2.6. Dependencies.....	8
3. Business Rules.....	8
4. Functional Requirements.....	9
4.1. Student and Faculty Users.....	9
4.2. SDS Administrators.....	11
5. Use Cases.....	12
5.1. Use Case 1.....	13
5.2. Use Case 2.....	14
5.3. Use Case 3.....	15
5.4. Use Case 4.....	16
5.5. Use Case 5.....	17
5.6. Use Case 6.....	18
5.7. Use Case 7.....	19
6. Data Requirements.....	20
6.1. Logical Data Models.....	20
6.2. Data Dictionary.....	21
6.3. Data Maintenance.....	25
7. External Requirements.....	25
7.1. User Interface.....	25
7.2. Software Interfaces.....	26
7.3. Communication Interfaces.....	26

8. Nonfunctional Requirements.....	27
8.1. Availability Requirements (Priority: 1).....	27
8.2. Compatibility Requirements (Priority: 5).....	27
8.3. Reliability Requirements (Priority: 2).....	27
8.4. Scalability Requirements (Priority: 3).....	27
8.5. Localization Requirements (Priority: 4).....	28
9. Reporting Requirements.....	28
9.1. Report 1.....	28
10. Supplemental Information.....	28
10.1. Frontend.....	28
10.2. Backend.....	29
10.3. Coding Practices.....	29
10.4. Additional Considerations.....	30
11. Glossary of Terms.....	30
12. Appendix.....	32
13. Approvals and Sign-off.....	32
13.1. etien1@umbc.edu.....	32

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) document is to define and outline the functional and non-functional requirements for Version 1.0 of the UMBC Interactive Accessibility Map. This document is intended to serve as a foundational guide for the development team of the project as well as a reference for all stakeholders. All requirements specified in this document are expected to be fully completed by the end of the semester.

1.2. Scope

An existing problem on the campus of UMBC is a lack of interactive mechanisms that aim to assist people with disabled movement to navigate around the campus. While the Office of Accessibility and Disability Services does have both digital and print-outs of accessibility maps on campus, they are not interactive nor do they provide any interior layout of elevators or other accessible tools within buildings. Therefore, the UMBC Interactive Accessibility Map is intended to be an interactive map that reads a user's location and calculates and displays the shortest path the user can take from one location to another, including accessibility concerns. The software will also help users to navigate within buildings and utilize existing interior mechanisms like elevators and doorways that are designed to be accessibly accommodating.

1.3. Intended Audience

The intended audience of this document is specifically aimed at the development team, project management, and project stakeholders. The development team of this project is composed of Benjamin Currie, Alexander Gallagher, and Jack Xu. The project management team is composed of Dr. Samit Shivadekar and his supporting Teaching Assistants and other staff. Additionally, Dr. Shivadekar is to be utilized as a major subject-matter expert throughout the development process of the UMBC Interactive Accessibility Map. Finally, the project stakeholders include Emily Tien, Stephanie Lazarus, and Tawny McManus of the Office of

Accessibility and Disability Services at UMBC. Primarily, Emily Tien will be the main stakeholder of contact throughout the development process of this project.

1.4. Definitions

- 1.4.1. Accessible: easily used or accessed by people with disabilities; adapted for use by people with disabilities

Merriam-Webster. (n.d.). Accessible. In *Merriam-Webster.com dictionary*. Retrieved September 22, 2025, from <https://www.merriam-webster.com/dictionary/accessible>

- 1.4.2. Interactive: involving the actions or input of a user, especially of, relating to, or being a two-way electronic communication system (such as a telephone, cable television, or a computer) that involves a user's orders (as for information or merchandise) or responses (as to a poll)

Merriam-Webster. (n.d.). Interactive. In *Merriam-Webster.com dictionary*. Retrieved September 22, 2025, from <https://www.merriam-webster.com/dictionary/interactive>

- 1.4.3. Map: a diagram or other visual representation that shows the relative position of the parts of something

Merriam-Webster. (n.d.). Map. In *Merriam-Webster.com dictionary*. Retrieved September 22, 2025, from <https://www.merriam-webster.com/dictionary/map>

1.5. References

- 1.5.1. Digital Accessibility Map:
<https://umbc.edu/wp-content/uploads/2024/12/2024-ACCESSIBLE-ROUTES-MAP-2.pdf>
- 1.5.2. General Location Map:

<https://umbc.edu/wp-content/uploads/2024/12/2024-CAMPUS-MAP-2.pdf>

1.5.3. Accessible Maps Information Page:

<https://accessibility.umbc.edu/campus-maps/>

2. Overall Description

2.1. Description

The Interactive Accessibility Map is designed to assist students who have disability concerns navigate the campus by providing an interactive map interface that performs best route calculations for them. By having the ability to input origin and destination points all throughout the UMBC campus, it enables students to be as efficient as possible while trying to navigate the otherwise challenging campus layout given their limited mobility. Additionally, the software provides some other information related to interior mechanisms, and attempts to assist students or faculty who are using it to locate the exact hallway they are attempting to enter. For an updated link to the development repository of the prototype, please refer to section 12.1 of the appendix.

2.2. User Classes

User Class	Description and Characteristics
Students	The primary intended users, particularly intended for those who are experiencing difficulties navigating the campus, are able to select from available location data to choose origin and destination locations on campus. They must be granted access to utilize this software based on the discretion of the Office of Accessibility and Disability Services at UMBC. If they want to submit feedback, there is a form they can fill out that emphasizes their concerns or suggestions for improvements to the software.
Faculty	Faculty who are experiencing any similar challenges with mobility also will be able to input information like a student would. Importantly, the faculty members would be able to input source and destination information, as well as having access to any interior information to help make

	their campus navigation easier.
Student Disability Services (SDS) Administrators	Administrators of this service would have the responsibility to manage access to the software by managing what students and faculty possess active copies of the software in use. Additionally, they have the responsibility of updating changes to the campus infrastructure that is reflected in future iterations of the software.

2.3. Operating Environment

- 2.3.1. The software should be accessible via a running computer device, as executable code in .exe format. Therefore, Windows and Linux devices will be able to utilize the software.
- 2.3.2. The software should be optimized for desktop devices.

2.4. Design and Implementation Constraints

- 2.4.1. The software must be able to handle multiple concurrent users using the service without performance degradation.
- 2.4.2. The software must maintain an accurate and updated database of geolocation information for the UMBC campus.
- 2.4.3. The software must present a professional appearance in the likeness of the UMBC color palette and aesthetic.

2.5. Assumptions

- 2.5.1. The software will be semi-regularly updated to include the most updated campus information, including path closures, building updates, and new geography.
- 2.5.2. The software will be automatically maintained to provide only students who should have access to the software with the ability to use it.

- 2.5.3. The software should be periodically rolled out with software updates and upgrades based on user feedback and recommended changes received through feedback mechanisms.

2.6. Dependencies

- 2.6.1. Library for the front-end library provided by GTK4, which we rely on for the development of our UI.
- 2.6.2. Miscellaneous utility C++ libraries that are used for back-end development of the program will function correctly.

3. Business Rules

Rule ID	Rule Definition	Type	Impacted Requirements
Navigation	Selecting the start and end point of a path must be easily interactive with a clear and intuitive design.	Implementation	4.1.1., 4.1.2., 4.1.3.
Valid Path	A path must not have its source equal to its destination.	Constraint	4.1.2.
Optimal Path	A path must be the most optimal route to take between the points possible, and in the event of ties, chooses one arbitrarily.	Constraint	4.1.2.
Map Integration	The map must be movable and interactive so that specific areas can be shown or analyzed with greater detail to make choosing locations more refined.	Implementation	4.1.3.
Saved Paths	Paths should be savable and have a name or other saved identity for quick navigation.	Implementation	4.1.4., 4.1.5.
Unique Saves	Each saved path must have a unique identifier associated with	Constraint	4.1.5.

	it, so no two paths appear the same.		
Accessibility Concerns	The app must comply with all major accessibility concerns as described by the requirements of the Office of Accessibility and Disability Services at UMBC.	Constraint	4.1.6.
Edit Paths	Administrators must have an interface to edit existing buildings and paths for future developmental changes.	Implementation	4.2.1., 4.2.2.
Feedback Mechanism	The app must possess a feedback mechanism that collects user feedback to be analyzed and implemented by developers and administrators.	Implementation	4.1.7.
Feedback Authentication	Users who submit feedback must have a valid UMBC email address.	Constraint	4.1.7.
Reachable Feedback	Feedback must be viewable by administrators that can be applied to the application in its next development cycle.	Implementation	4.2.3.

4. Functional Requirements

4.1. Student and Faculty Users

4.1.1. Users should be able to see and interact with the map of the UMBC campus. They should also be able to see:

- A search bar for source and destination points.
- An option to review and select from existing saved paths.
- A form location to submit feedback.

4.1.2. Users should be able to intuitively see how to select a start and end point with the following options:

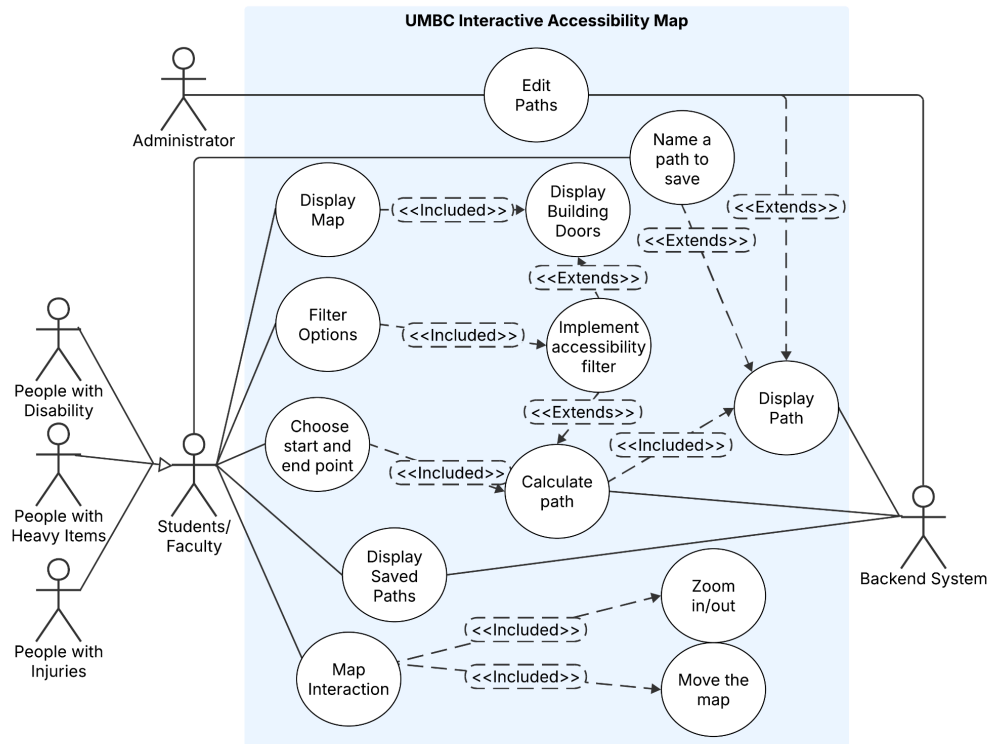
- There should be interactive points on the map to select what building you want to choose and all the doors for the building should be shown.
 - A search bar of options should be available for start and end points that can be typed into, searched through, and displayed.
- 4.1.3. Users should be able to interact with the map, either by zooming in and out or panning across the map around so the focal point can be changed.
- 4.1.4. Users should be able to click on the saved paths button, which brings them to the saved paths screen. This screen includes a search tool and dropdown menu of the user's saved paths.
- 4.1.5. Paths can be loaded from saves to display a previously computed route, or they can be saved with a name to be used again in the future.
- 4.1.6. Path-finding must consider only accessible routes throughout the campus, applied through an accessibility filter. These requirements can be listed as:
- Avoiding stairs and non-accessible paths like steep ramps (for more information, see the glossary on how we define “steep”).
 - Avoiding interior obstructions, ledges, or steps.
 - Avoiding closed routes, such as those under construction.
 - Including elevators, non-steep ramps, and mechanical entranceways.
 - Including non-obstructed interior floor plans and flat ground.
 - Including major ingress and egress points to choose as starting or destination locations.
- 4.1.7. Users can access a form page to submit feedback about how they want the application to be updated or changed. This page includes:
- UMBC Authentication, requiring users to be logged in to a UMBC email account.

- General multiple-choice selection questions asking how satisfied users are with existing maps.
- Open-ended free response questions that include how the user wants to improve the application, whether an implemented piece of geographic information was incorrect or outdated, and other concerns about the application's accessibility.

4.2. SDS Administrators

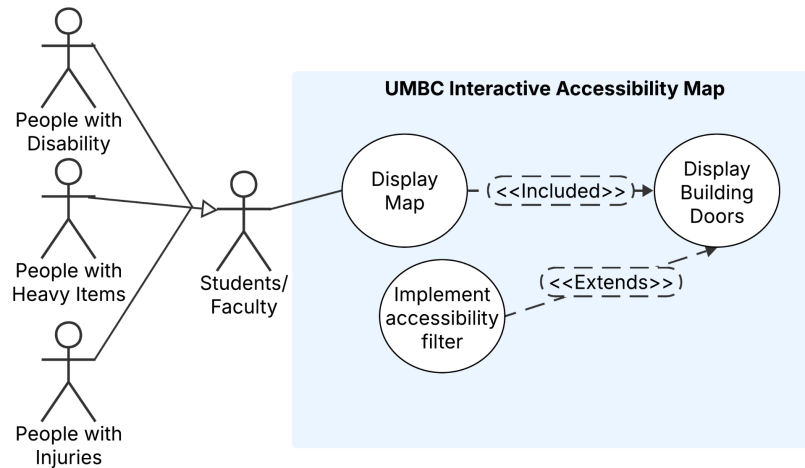
- 4.2.1. Administrators must be able to manipulate the map's geographic information, removing or altering the state of existing paths.
- 4.2.2. Administrators must be able to add or design new paths whenever they are developed.
- 4.2.3. Administrators must be able to access and read user feedback to see how the application is being used and to adjust accordingly.

5. Use Cases



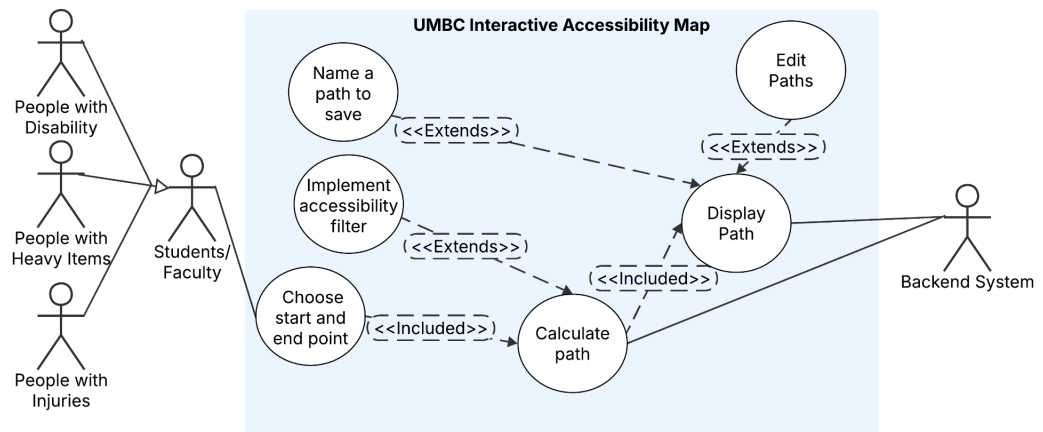
The UMBC Interactive Accessibility Map has 2 primary actors that can interact with the application: Students/Faculty and Administrators. From top to bottom, the Administrators' main role is to update and edit the paths that students can take by either closing paths for construction or adding new paths that can be made through the command line. Students or faculty can see the map of the campus and zoom in or out as well as pan across the map to find the buildings they are looking for. Building selection is made by clicking on the indicated dots of the building or by typing the building name in the search bar. Once the student or faculty member selects their start and end points, the map will show the most optimal path they need to take from that source to that destination. At this point, students/faculty can choose to save this path for later use, or choose to access all their saved paths by clicking on the saved path button.

5.1. Use Case 1



Name (description)	Displays UMBC Campus
Requirement ID	4.1.1.: Users should be able to see and interact with the map of the UMBC campus.
Actors	Students/Faculty
Secondary Actors	Backend System
Pre-conditions	The app is open and actively running.
Trigger	N/A
Post-conditions	The UMBC campus is displayed with door outlines.
Main Success Path (Primary Flow)	Students/faculty can see the UMBC campus and building doors.
Actor Actions	System Response
Student opens the app	Application displays the campus

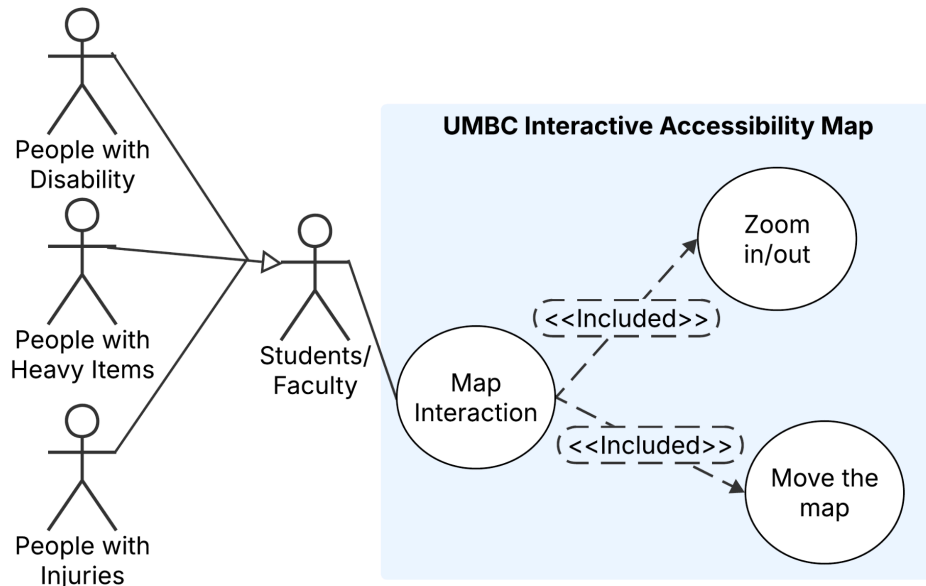
5.2. Use Case 2



Name (description)	Find a path from building to building
Requirement ID	4.1.1.: Users should be able to see and interact with the map of the UMBC campus. 4.1.2.: Users should be able to intuitively see how to select a start and end point.
Actors	Students/Faculty
Secondary Actors	Backend System
Pre-conditions	A start and end point have been selected.
Trigger	Click the go button.
Post-conditions	The path from building to building is displayed with filters selected.
Main Success Path (Primary Flow)	Students/faculty select a start and end point along with the filters they want and the path between the points is shown.
Actor Actions	System Response
Select or type the starting point	Stores the start point
Select or type the end point	Stores the end point
Choose the filter for the path calculation	Store the filters selected to be used for path calculations

Click go	Calculates the path between the points and displays the path
----------	--

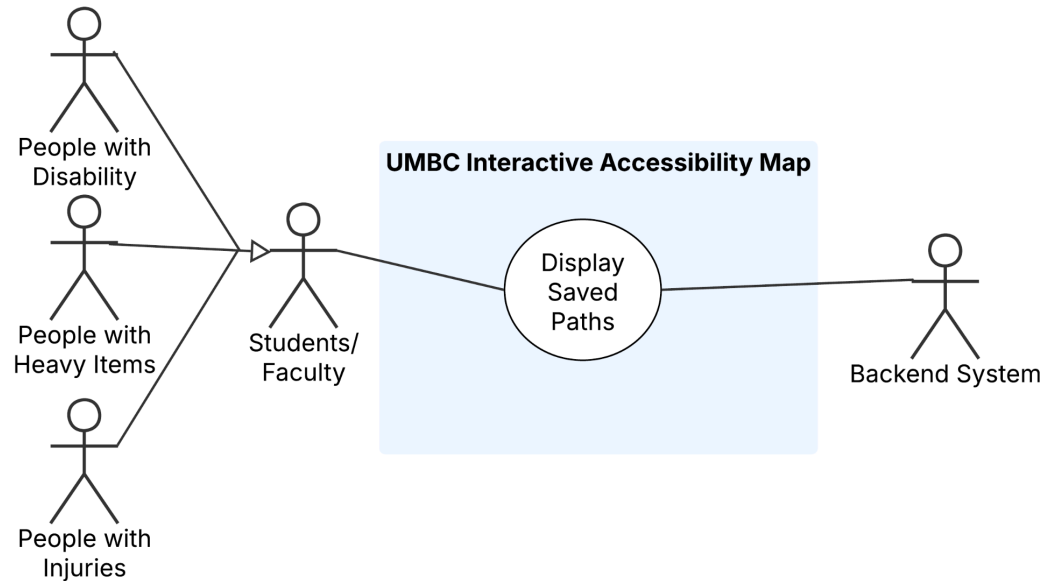
5.3. Use Case 3



Name (description)	Interacting with the map
Requirement ID	4.1.3.: Users should be able to interact with the map, either by zooming in and out or panning across the map around so the focal point can be changed.
Actors	Students/Faculty
Secondary Actors	Backend System
Pre-conditions	The map is displayed.
Trigger	Hand, + sign, or - sign is clicked.
Post-conditions	The map is changed according to what was clicked either zoomed in/out or moving the focal point.
Main Success Path (Primary Flow)	Student/faculty clicks on 1 of 3 buttons and the map will change according to what button is clicked.
Actor Actions	System Response

Either click on the zoom in or zoom out buttons, or use mouse to move around	Changes what the map looks like based on what is pressed: either zooming in or out, or moving the focal point of the map
--	--

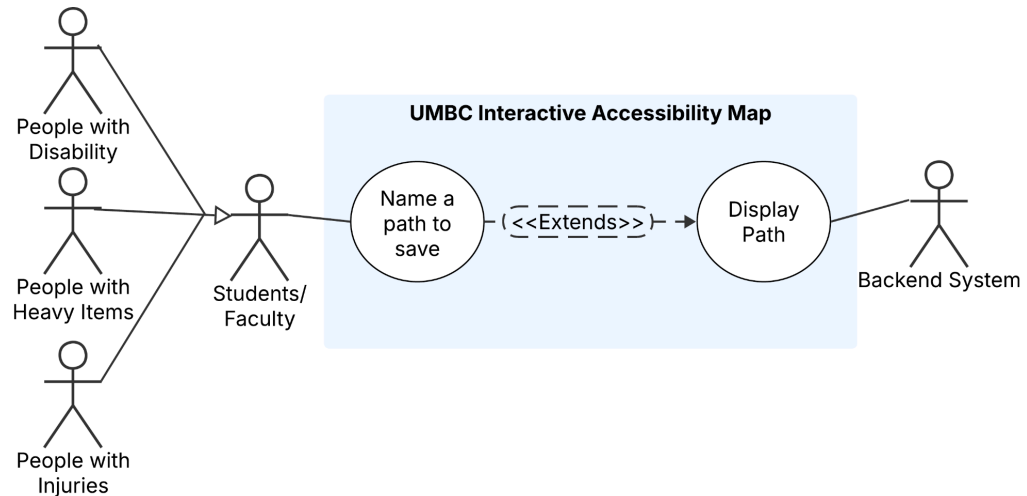
5.4. Use Case 4



Name (description)	Displays saved paths
Requirement ID	4.1.4.: Users should be able to click on the saved paths button, which brings them to the saved paths screen. This screen includes a search tool and dropdown menu of the user's saved paths.
Actors	Students/Faculty
Secondary Actors	Backend System
Pre-conditions	The program is actively running.
Trigger	Clicks on the open saved paths button.
Post-conditions	Shows the saved paths of the user that they can choose from.
Main Success Path (Primary Flow)	Click on the open saved paths and all the saved paths of the user are displayed and

	can be selected from.
Actor Actions	System Response
Click on the open saved path button	Opens a screen that displays the saved path of the user

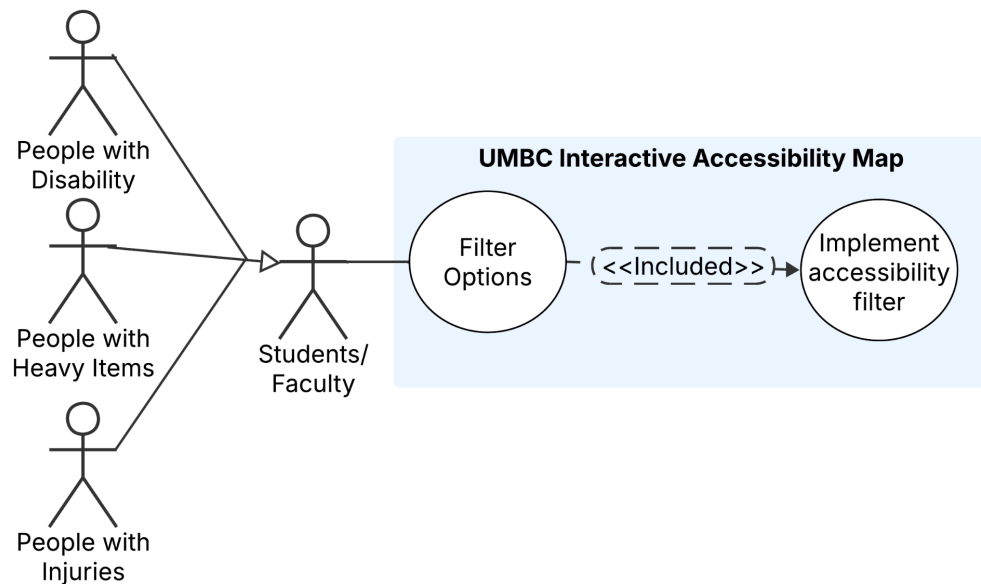
5.5. Use Case 5



Name (description)	Save a path
Requirement ID	4.1.5.: Paths can be loaded from saves to display a previously computed route, or they can be saved with a name to be used again in the future.
Actors	Students/Faculty
Secondary Actors	Backend System
Pre-conditions	A path has been created.
Trigger	Click on the save button.
Post-conditions	The path gets saved with a name.
Main Success Path (Primary Flow)	The path that was created is saved and given a name.
Actor Actions	System Response
Click on the save button	Shows the screen where you can give the

	path a name
Type in a name	Stores that path with the given name to the database

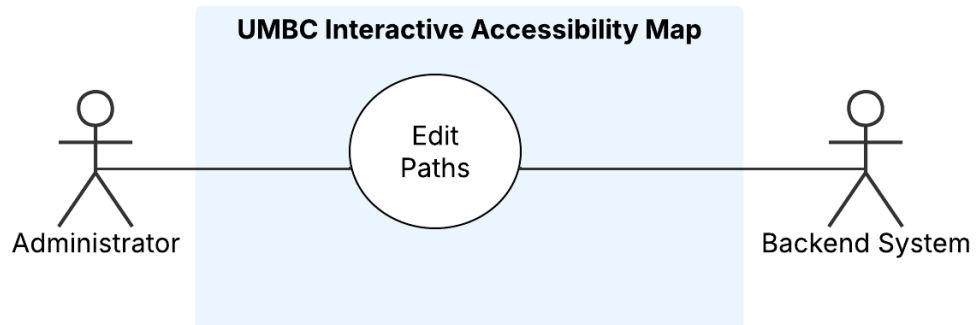
5.6. Use Case 6



Name (description)	Select filter options
Requirement ID	4.1.6.: Path-finding must consider only accessible routes throughout the campus, applied through an accessibility filter.
Actors	Students/Faculty
Secondary Actors	Backend System
Pre-conditions	The program is actively running.
Trigger	Toggle on or off the accessibility filter.
Post-conditions	What is displayed or the navigation filter is changed according to what filter is selected.
Main Success Path (Primary Flow)	Users toggle on or off a filter and the map or navigation is changed according to the filter.

Actor Actions	System Response
Toggles on/off the filter (avoid stairs, avoid interior, show mechanical doors)	The map of navigation is changed to what filter is selected

5.7. Use Case 7



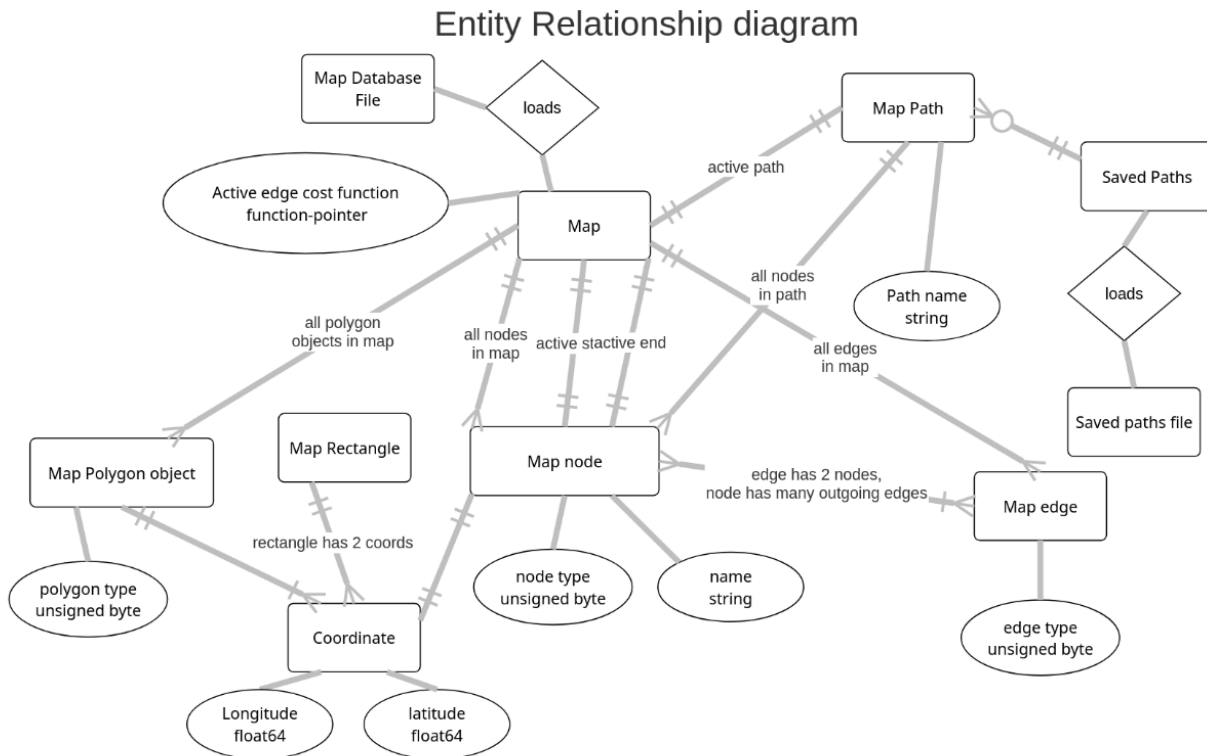
Name (description)	Edit paths
Requirement ID	4.2.1.: Administrators must be able to manipulate the map's geographic information, removing or altering the state of existing paths. 4.2.2.: Administrators must be able to add or design new paths whenever they are developed.
Actors	Administrators
Secondary Actors	Backend System
Pre-conditions	Command-line tool to edit the software is open.
Trigger	Typing commands and coding software to edit the paths/buildings.
Post-conditions	The edits to the buildings and path are shown on the map.
Main Success Path (Primary Flow)	Commands to edit the buildings or path are written and manipulated to edit the existing map.

Actor Actions	System Response
Enters a command to edit paths/buildings	Makes changes to the map according to the command entered

6. Data Requirements

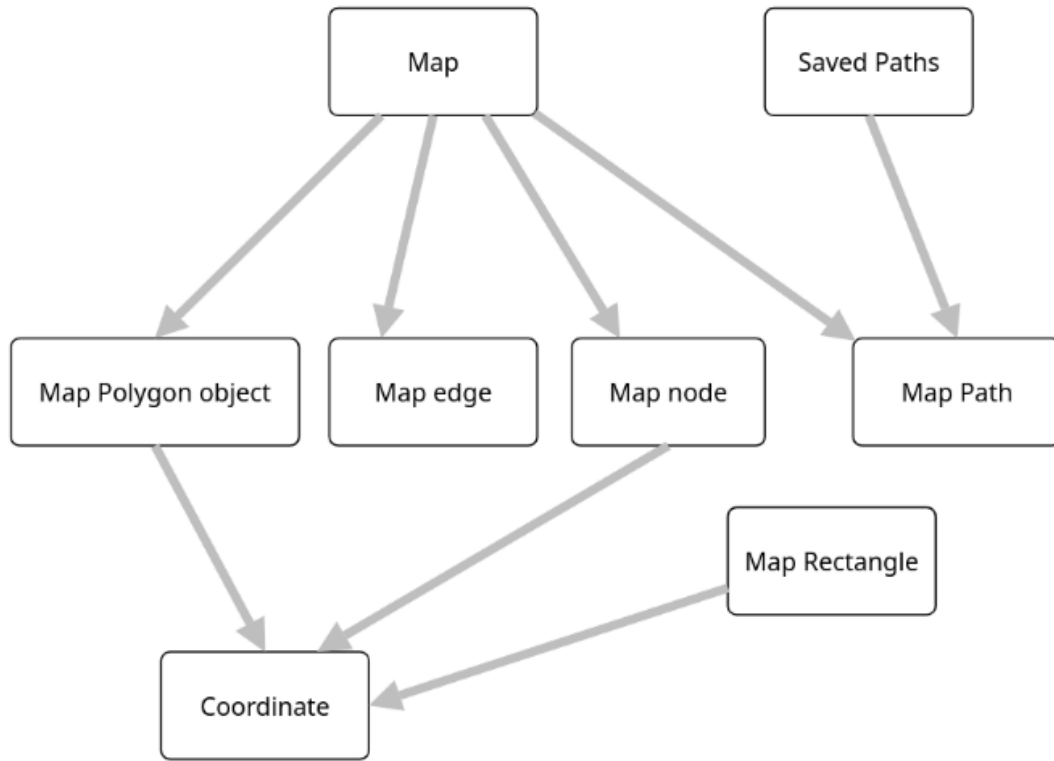
6.1. Logical Data Models

6.1.1. Entity-Relationship Diagram



6.1.2. Entity Ownership Diagram

Entity Ownership diagram



6.2. Data Dictionary

Attribute	Entity	Description	Required	Type	Source Database	Length	Default Value	Comments
Longitude	Coordinate	Longitudinal coordinate	Yes	Float64	Graph Database File	N/A	0.0	Actual coordinates of objects
Latitude	Coordinate	Latitude Coordinate	Yes	Float64	Graph Database File	N/A	0.0	Actual coordinate of objects
Poly-Coordinates	Map polygon object	Polygon to make shapes of building, lakes, fauna,	Yes	Array	Graph Database File	N_coords which is how many corners on	N/A	Simple basic polygons

		etc				the polygon		
Polygon type	Map Polygon object	The object type for the polygon	Yes	Unsigned byte	Graph Database File	N/A	Building	What the polygon describes from an abstract point of view
Map-rect bottom left	Map Rectangle	The bottom left corner of the rectangle	Yes	Coordinate	Computed upon startup	N/A	0,0	Used for rendering purposes
Map-Rect Top right	Map Rectangle	The top right corner of the rectangle	Yes	Coordinate	Computed upon program startup	N/A	0,0	Used for rendering purposes
Map-active start	Map	The currently active start position for the path search. Will be used by a path finding algorithm.	Yes	Map node	Selected at runtime of program	N/A	NULL	User selected
Map-active end	Map	The currently active end position for the path search. Will be used by a path finding algorithm.	Yes	Map node	Selected at runtime of program	N/A	NULL	User selected
Active edge cost function	Map	How edge weight cost will be calculated by the path finding algorithm.	Yes	Function pointer which takes edge as parameter	N/A	N/A	NULL	This is set based on the active search preferences. For example, avoiding sidewalks

								will impact which edge cost function is used.
Active path	Map	The active path which was found as a result of the pathfinding algorithm. This will be rendered.	Yes	Map-Path	Calculated at runtime	N/A	NULL	Will not be null once a path has been set.
All nodes	Map	An array of all nodes for the map	Yes	Array of nodes	Graph Database File	N_nodes which is the total nodes required to describe the graph.	N/A	
All edges	Map	An array of all edges for the map	Yes	Array of edges	Graph Database File	N_edges which are the total number of edges required to describe the graph.	N/A	
All map polygon objects	Map	Array of all map polygon objects	Yes	Array of map polygon objects	Graph Database File	N_polygon objects	N/A	
Node coordinate	Node	The coordinate of a map node	Yes	Coordinate	N/A	N/A	0,0	
Node name	Node	The name of a location	Yes	String	Graph Database File	Length of string	NULL	Most nodes will not have a name. Only notable

								locations
Node type	Node	The type of node	Yes	Unsigned byte	Graph Database File	N/A	NODE_TYPE_JOIN	Most nodes are used to join edges but some are actually notable locations on campus
Node edges	Node	The outgoing edges connected the node	Yes	Array	Graph Database File	The number of outgoing edges		
Edge start	Edge	The pointer to the first node as part of the edge.	Yes	Node	Graph Database File	N/A	NULL	
Edge end	Edge	The pointer to the second node as part of the edge.	Yes	Node	Graph Database File	N/A	NULL	
Edge type	Edge	The type of edge	Yes	Unsigned byte	Graph Database File	N/A	SIDE WALK	Can be one of these: sidewalk, road, stairs, ramp, or hallway
Path nodes	Path	The node sequence that make up a path	Yes	Array	Saved paths file or computed at runtime	N nodes which is the length of the path	N/A	The nodes have an order.
Path name	Path	Paths can be named so that they are easy to find when saved	No	String	Saved paths file	Length of string	NULL	This is an extra feature
Saved paths paths	Saved paths	The array paths which	No	Array	Saved paths file	Number of saved	NULL	This is an extra feature

		are from the saved paths file				paths		
--	--	-------------------------------	--	--	--	-------	--	--

6.3. Data Maintenance

- 6.3.1. The user's saved paths must be stored as long as the program is properly installed and functions on the host computer.
- 6.3.2. The user's accessibility choice is stored until it is toggled whenever the user wishes to change their map accessibility preferences.
- 6.3.3. The user's feedback data is stored indefinitely in the cloud to be accessed by administrators who will handle the form data accordingly.
- 6.3.4. In the event of an intended database transfer, the data will be stored in a fashion that is backwards compatible with encoding into a different scheme on another platform. In this case, Python scripts will be used to transfer the information and re-encode the data into the new scheme.

7. External Requirements

7.1. User Interface

- 7.1.1. The application must contain a help icon or feature that helps explain how to use the application, including basic controls and how to use the path-finding features.
- 7.1.2. The application must accept both mouse and keyboard inputs to navigate all aspects of its functionality, from navigation to searching to saving paths.
- 7.1.3. The application should have intuitive and colorful paths and icons that will help users differentiate between what each shape and color means.

- 7.1.4. The application should have large font sizes, with all text in it being compatible with screen readers and other text/copyable components.
- 7.1.5. The application should have an intuitive and clear design as indicated in 12.2, a rough sketch of how the expected output is to appear.

7.2. Software Interfaces

- 7.2.1. All paths must be created and processed before being saved.
- 7.2.2. Saved paths will be stored and read from the external “.save” file that runs alongside the program.
- 7.2.3. All edges and nodes will be stored and pulled from the “.map” file that runs alongside the program.
- 7.2.4. The map and its images will be pulled from either PNG or JPEG-formatted files to be displayed to the user.
- 7.2.5. Optimal path calculation will pull data from the “.map” file to see all the available nodes and edges that could be taken and generate the most optimal route possible.

7.3. Communication Interfaces

- 7.3.1. Any map updates implemented by the administrators will be announced to the user.
- 7.3.2. A link to a UMBC-authenticated Google Form will be available to collect user feedback.
- 7.3.3. A link to the phone number and email for the Facilities Work Control (the department in control of repairs to accessibility-related mechanisms on the UMBC campus) will be posted and displayed so a user can report broken doors and elevators directly to the people who will fix it.

8. Nonfunctional Requirements

8.1. Availability Requirements (Priority: 1)

- 8.1.1. The application must work every time the user launches the application.
- 8.1.2. The application must never crash when launched.
- 8.1.3. The application must have offline capabilities, and can run without needing Internet connection.

8.2. Compatibility Requirements (Priority: 5)

- 8.2.1. The application must be compatible with Windows and Linux machines.

8.3. Reliability Requirements (Priority: 2)

- 8.3.1. The application must update whenever any changes are made to either the paths or buildings in the application.
- 8.3.2. All path calculations must be fast and show no visible delay when calculated.
- 8.3.3. The application should never have a memory leak.

8.4. Scalability Requirements (Priority: 3)

- 8.4.1. The application must be able to handle a larger map size, or count of nodes and edges.
- 8.4.2. The algorithms governing path calculation and door placement must be able to handle changes to map size.
- 8.4.3. The number of saved paths you can have must be large enough so that no user will run out of space when attempting to store more.

8.5. Localization Requirements (Priority: 4)

- 8.5.1. The system must handle any search queries in English, but the saved names of paths can be in any language.
- 8.5.2. The application needs to be compatible with screen readers, so that users who are visually impaired are able to navigate efficiently just like other users.

9. Reporting Requirements

9.1. Report 1

- 9.1.1. Report Title is Software Requirement Specifications
- 9.1.2. Headers contain the report title, sponsor's name, and the date range of Sprint 1 separated by dashes
- 9.1.3. Footers show the current page number
- 9.1.4. A table of contents is included at the beginning of the report to quickly navigate its various sections

10. Supplemental Information

10.1. Frontend

Our frontend will use GTK4 to render buttons, the map, and all other interface elements. We will be designing our own map user-interface written on top of the Cairo graphics API. We will be pulling our map image from the current map of UMBC located at appendix 12.3, which we will draw nodes, edges, and lines on top of. In the event of changes to campus geography, future maps will be pulled from similar official sources provided by UMBC. The inspiration we have had for the expected output of graph drawing will be similar to appendix 12.4, which is the official UMBC Accessibility Map that shows all major routes for pedestrians

to take across campus. However, the expected appearance of the application will be more like appendix 12.2, of which we will draw upon the map like that of 12.4.

10.2. Backend

Any notable locations that could be considered as passageways, entrances, exits, or buildings will be represented as a series of nodes, edges, and polygons. This map data is stored in a map file “.map” which is a custom binary encoding of all the codified data in the map, excluding images. Notable locations may be accompanied by an image for ease of use and to increase map clarity. These images will be stored in an assets folder, which may be referenced by map data in the event of a user clicking or hovering over a respective location. Additionally, users will have the option to save any path they have created. These saved paths will also be stored in a separate “.save” file. These saved paths will store a name and the corresponding node-sequence to reach the location. All of these supporting files and folders will be stored locally on the user’s machine along with the application program.

10.3. Coding Practices

For good practice in our application design, our code should be as readable as possible with variables and function names being very descriptive and obvious with their application. Function parameters should be ordered in a consistent and common-sense manner. Every function must include a simple description of what it does and provide any additional information to explain any parts of it that are confusing or difficult to understand. Functions with similar goals or ones that operate on similar objects should be grouped together to avoid the need of searching around the entire codebase. All code should be stylistically similar throughout the entire program. Use of static functions and namespaces will be encouraged in .cpp files to avoid namespace pollution and to hide any complicated but necessary subroutines. The code should be efficient, fast, and memory-leak free with various tests and memory checks in place throughout the code.

10.4. Additional Considerations

Pop-up windows will be occasionally used to prompt the user to fill in some dialogue boxes. The pop-up box might include fields for path names or to show all the saved paths saved on the user's system. These popups will have exit or "X" buttons to close out of in the case that the user made an accidental click. Also, a help button will be included on the main application page that will explain the basics of some controls included in the program as well as how to use its various components. Importantly, a link will be made obvious that will direct the user to a UMBC-authenticated Google Form, of which they can respond to for feedback to improve the software.

11. Glossary of Terms

- 11.1. SDS: Student Disability Services. The UMBC administrative body that oversees current accessibility and disability concerns on campus, and would be the administrators of this accessibility map.
- 11.2. Stakeholder: An individual or group of people (i.e. Mrs. Emily Tien, Prof. Samit Shivadekar, the development team) with interest or concern with the outcome of the project.
- 11.3. Pathing Algorithm: The core computational method (in this case, A* search) used to calculate the optimal path between selected nodes.
- 11.4. Node: A single, quantifiable point on the map (i.e. a door, hallway intersection, or corner) used by the pathing algorithm to build routes.
- 11.5. Edge: A connection between two nodes representing a path segment (i.e. a sidewalk, ramp, or hallway) stored in the .map file.
- 11.6. Optimal Path: The shortest or most efficient route between a source and destination as determined by the pathing algorithm, respecting all accessibility concerns.

- 11.7. Steep: Any path segment (edge) with an incline that exceeds 20 degrees. Paths classified as steep will not be included in the routing process, considering accessibility concerns.
- 11.8. Obstruction: Any path segment (edge) that is temporarily closed due to construction, maintenance, or some other barrier that is to be updated by SDS Administrators.
- 11.9. Accessibility Filter: The user-controlled setting that enforces accessibility concerns during path calculation.
- 11.10. UMBC Authentication: The process of requiring a user to log in with a valid UMBC email address and password, primarily used to authorize access to the feedback form.
- 11.11. Frontend: The client-side part of the application, developed using GTK4 and Cairo, responsible for rendering the map, UI, and handling user interaction.
- 11.12. Backend: The server-side logic and data management responsible for storing map data in the .map file and saved paths in the .save file, as well as performing optimal path calculations.
- 11.13. GTK4: The external cross-platform toolkit library used to develop the application's frontend user interface.
- 11.14. Geographic Information: The digital representation of the UMBC campus's physical features, including the coordinates, types, and connectivity of all nodes, edges, and polygons.
- 11.15. Map Polygon Object: A multi-sided shape used to define the boundaries of large features on the map, such as buildings, lakes, or fields.

12. Appendix

12.1. GitHub Repository: <https://github.com/AG446/CMSC-447-Project>

12.2. Low-Fi Wireframe:



12.3. Map of UMBC Campus:

<https://umbc.edu/wp-content/uploads/2024/12/2024-CAMPUS-MAP-2.pdf>

12.4. Map of Exterior Accessible Routes on UMBC Campus:

<https://umbc.edu/wp-content/uploads/2024/12/2024-ACCESSIBLE-ROUTES-MAP-2.pdf>

13. Approvals and Sign-off

By signing this document, the customer and developers agree to the details specified in this document, specifically: the scope of the project, the functional requirements, and the non-functional requirements.

13.1. etien1@umbc.edu

Name	Position	Reviewed	Signature	Date
Emily Tien	Stakeholder	10/6/2025	Emily Tien	10/6/2025